

# Entender las salidas de los comandos

DNS · Shell · Amazon EC2 — Camilo Aguirre & Juan David Rangel

## CONTENIDO

1. Cliente vs servidor: el concepto que más confunde
2. ¿Cuál es la idea del lab?
3. DNS — comando por comando
4. Transferencia de zona (replicación) en 2 PCs
5. Shell — comando por comando
6. EC2 — comando por comando
7. Tabla resumen: "¿por qué veo esto?"

## 0. Cliente vs servidor: el concepto que más confunde

En este lab **cada máquina es las DOS cosas al mismo tiempo**: corre un servidor DNS y actúa como cliente.

| Rol                           | Dónde vive la config                                       | Qué significa                              |
|-------------------------------|------------------------------------------------------------|--------------------------------------------|
| <b>Cliente</b><br>(pregunta)  | <code>/etc/resolv.conf</code> o el adaptador (Windows)     | "A qué servidor le pido que resuelva"      |
| <b>Servidor</b><br>(responde) | <code>/etc/named.conf</code> + zonas, o rol DNS de Windows | "De qué zonas soy dueño y qué datos tengo" |

**Regla 1:** `nameserver 127.0.0.1` = "me pregunto a mí mismo" → prueba del **punto 8** (en el propio servidor).

**Regla 2:** poner la **IP de otra máquina** como resolver = prueba del **punto 7** (un cliente usando nuestro DNS).

### ¿Por qué a veces responde "el local" y no "el servidor DNS"?

Porque en esa prueba **el local ES el servidor DNS**. No son máquinas distintas: es la misma.

- Para `camilo.org.uk`, Slackware es **maestro** → dueño de los datos.
- Para `juan.com.it`, Slackware es **esclavo** → BIND guardó una **copia completa** (transferida una vez) y la sirve como autoritativa. Un esclavo **NO reenvía** la consulta al maestro: responde con su copia.
- Por eso, al preguntar desde Slackware, **no sale a la red**: ya tiene la respuesta adentro.

`nslookup` muestra **a quién le preguntaste**, no "de dónde vinieron los datos":

```
Slackware (misma máquina):  Server: 127.0.0.1      ← me pregunté a mí mismo
Windows GUI (otra máquina): Server: 192.168.82.10   ← le pregunté a Slackware
```

# 1. ¿Cuál es la idea del lab?

Montar un **servicio DNS de empresa** y demostrar que resuelve. Concretamente:

1. **Dos dominios reales** ( `juan.com.it` , `camilo.org.uk` ) con primario + secundarios.
2. **Redundancia**: si un servidor cae, otro responde igual.
3. Que resuelva **hacia adentro** (tus dominios) **y hacia afuera** ( `google` , `escuelaing` ).
4. Que **un cliente distinto del servidor** lo use y resuelva (punto 7).

**Por qué cada máquina es cliente y servidor:** en una empresa los *secundarios* existen para dar redundancia. El lab no da VMs "de cliente" aparte, entonces usa las mismas máquinas en los dos roles. De ahí sale toda la confusión del `127.0.0.1`.

El **punto 8** ("¿funciona en el servidor? ¿por qué?") es un *control* puesto a propósito: te obliga a ver que un servidor también es cliente de sí mismo, y que su propio resolver (y su IP) tienen que estar bien configurados. Por eso la guía pide **mostrar la IP final del servidor**.

**Resumen:** la idea NO es "que contesta el local". La idea es: *hay un servicio DNS con redundancia, que resuelve adentro y afuera, y lo demuestra un cliente que NO es el servidor*. La prueba que más pesa es la de la **Windows GUI apuntando a 192.168.82.10**.

## 2. DNS — comando por comando

**nslookup srv1.camilo.org.uk**

```
Server: 127.0.0.1
Address: 127.0.0.1#53

Name: srv1.camilo.org.uk
Address: 10.2.78.75
```

**Qué ves:**

- **Primer bloque** = *quién contestó*. `127.0.0.1` es el `named` local. El `#53` es el puerto.
- **Segundo bloque** = *la respuesta*: el registro **A** (nombre → IPv4).

**Por qué:** el resolver está en `127.0.0.1`, así que la consulta fue al servidor local, que es autoritativo y responde de memoria.

Si aparece `Non-authoritative answer`, quien contestó **no es dueño** de la zona y respondió desde **caché** (no es falso: es una copia reciente). Cuando contesta el dueño, esa línea no aparece.

**nslookup srv1.juan.com.it (desde Slackware)**

```
Name: srv1.juan.com.it
Address: 10.2.78.74
```

**Por qué funciona sin preguntarle a Solaris:** Slackware es **esclavo** de `juan.com.it`. Un esclavo que carga una zona la sirve como autoritativa. Ese es el objetivo de la replicación: cualquiera de los servidores puede responder, incluso si el primario está caído.

**nslookup www.camilo.org.uk (un alias)**

```
www.camilo.org.uk canonical name = srv1.camilo.org.uk.
Name: srv1.camilo.org.uk
```

Address: 10.2.78.75

**Por qué salen DOS cosas:** `www` es un **CNAME** (alias). El servidor devuelve el nombre canónico **y** el A del destino, en el mismo mensaje.

### **nslookup www.google.com (dominio externo)**

Con internet, aparece normalmente `Non-authoritative answer`.

**Por qué:** el `named` local **no es dueño** de `google.com`, así que actúa de **recursor**: consulta las raíces (`named.ca`), luego el TLD `.com`, luego Google. Guarda en **caché** y entrega. Al venir de caché → "no autoritativa".

### **nslookup -type=NS juan.com.it**

```
juan.com.it nameserver = dns1.juan.com.it.
```

Se pidió el registro **NS** (quién es autoritativo), no una IP.

### **nslookup -type=MX juan.com.it**

```
juan.com.it No answer
```

**Por qué NO es un error:** el nombre **existe** pero no tiene registro de ese **tipo** (MX = correo, comentado en la zona).

| Resultado              | Significa                                                       |
|------------------------|-----------------------------------------------------------------|
| <code>No answer</code> | El nombre existe, ese tipo de registro no                       |
| <code>NXDOMAIN</code>  | El nombre <b>no existe</b> (p. ej. contra el DNS de la escuela) |

### **set debug**

Baja el detalle al máximo y muestra el **paquete DNS completo**: `QUESTIONS` (qué se preguntó, con tipo **A y AAAA**), `ANSWERS`, flags y SOA. Es la radiografía de la consulta.

### **cat /etc/named.conf**

| Bloque | Significa |
|--------|-----------|
|--------|-----------|

|                                                                  |                                                       |
|------------------------------------------------------------------|-------------------------------------------------------|
| <code>options { directory "/etc/DNS"; }</code>                   | Dónde están los archivos de zona                      |
| <code>zone "." { type hint; file "named.ca"; }</code>            | <b>Raíces:</b> sin esto no resuelve dominios externos |
| <code>zone "camilo.org.uk" { type master; }</code>               | <b>Soy el dueño</b> de esta zona                      |
| <code>zone "juan.com.it" { type slave; masters { ... }; }</code> | <b>Copio</b> esta zona de otro servidor               |
| <code>allow-transfer { ... }</code>                              | Solo estas IPs pueden pedir la copia (AXFR)           |
| <code>notify yes;</code>                                         | Avisa a los esclavos apenas cambia la zona            |

**ps -ef | grep named**

```
named 514 1 0 21:02 ? /usr/sbin/named -u named
```

- `514` = **PID**.
- `1` = **proceso padre** (init): lo levantó el sistema, no un usuario.
- `-u named` = **bajó privilegios**: corre como usuario `named`, **no como root**. Seguridad.

**svcs dns/server (Solaris)**

```
STATE    STIME    FMRI
online   ...      svc:/network/dns/server:default
```

`online` = corre **ahora**; `enabled=true` = arranca **al boot**. Son cosas distintas, y el punto 9 pide lo segundo.

**nslookup en Windows → Server: UnKnown**

```
Server:  UnKnown
Address: 192.168.82.10
```

**Por qué dice "UnKnown":** Windows intenta mostrar el *nombre* del servidor DNS y para eso consulta el inverso (PTR) de la IP. Como no hay registro inverso → no lo puede

nombrar. **Es cosmético, no un error.**

## Comandos de Windows útiles

```
Get-DnsClientServerAddress -InterfaceAlias "Ethernet 2" -AddressFamily IPv4  
# → a qué servidor DNS apunta el cliente (lado "cliente")
```

```
Get-DnsServerZone -Name "juan.com.it"  
# → ZoneType: Secondary (lado "servidor": es esclavo de esa zona)
```

```
Resolve-DnsName srv1.juan.com.it -Type A  
# → equivalente moderno de nslookup
```

### 3. Transferencia de zona (replicación) en 2 PCs

Un **esclavo** no inventa los datos: le pide la zona al **maestro** y guarda la copia. Eso es la *transferencia de zona*, y es lo que permite que el esclavo responda aunque el maestro esté caído (la redundancia del lab).

#### Las 3 piezas que la hacen funcionar

| Pieza                                              | Dónde                | Qué dice                                                |
|----------------------------------------------------|----------------------|---------------------------------------------------------|
| <code>type master</code> / <code>type slave</code> | En cada zona         | Si soy el dueño o el que copia                          |
| <code>allow-transfer { ... }</code>                | En el <b>maestro</b> | <b>A quiénes</b> les permito pedir la copia (seguridad) |
| <code>masters { ... }</code>                       | En el <b>esclavo</b> | <b>De quién</b> copio la zona                           |

#### Cuándo transfiere: el serial

El esclavo revisa el **serial** del SOA cada `Refresh` (o enseguida si el maestro manda `notify`). Si el serial del maestro es **mayor**, pide la zona (**AXFR** = completa, **IXFR** = incremental). Si el serial **no sube**, no transfiere. Verificación: comparar el serial con `nslookup -type=SOA` en maestro y esclavo — debe ser el mismo.

**El detalle de las 2 PCs:** el `intnet` de VirtualBox es **local de cada host** — no cruza entre computadoras. Con las VMs repartidas en 2 PCs, la transferencia tiene que viajar por la **red física de la uni** ( `10.2.78.x` ). Por eso se agregaron esas IPs a `allow-transfer` / `masters` .

#### Configuración actual (lista para 2 PCs)

| Servidor  | Zona                       | Rol     | Quién puede transferir / de quién                                                     |
|-----------|----------------------------|---------|---------------------------------------------------------------------------------------|
| Slackware | <code>camilo.org.uk</code> | maestro | <code>allow-transfer { 192.168.82.11; 192.168.82.12; 10.2.78.75; 10.2.78.77; }</code> |

|                 |               |            |                                                                                |
|-----------------|---------------|------------|--------------------------------------------------------------------------------|
| Solaris         | juan.com.it   | maestro    | allow-transfer { 192.168.82.10;<br>192.168.82.13; 10.2.78.74;<br>10.2.78.76; } |
| Slackware       | juan.com.it   | esclavo    | masters { 192.168.82.11;<br>10.2.78.75; }                                      |
| Solaris         | camilo.org.uk | esclavo    | masters { 192.168.82.10;<br>10.2.78.74; }                                      |
| Windows<br>GUI  | juan.com.it   | secundario | MasterServers 192.168.82.11;<br>10.2.78.75                                     |
| Windows<br>Core | camilo.org.uk | secundario | MasterServers 192.168.82.10;<br>10.2.78.74                                     |

**Regla práctica:** cualquier par de máquinas que quede en **PCs distintas** tiene que hablarse por `10.2.78.x`. Por eso se dejan *ambas* IPs en la config: funciona con las VMs repartidas en 2 PCs o todas en una sola.

## 4. Shell — comando por comando

**schedult-task-script.sh \* \* \* \* \*** y luego **crontab -l**

```
* * * * * /usr/local/bin/mi-script.sh
```

**Por qué 5 asteriscos:** son los 5 campos de tiempo de cron:

```
minuto  hora  día-mes  mes  día-semana  comando
```

`* * * * *` = "todos los minutos". Por eso el script corre solo.

**Ojo Solaris:** el cron viejo **no soporta** `*/N`; usar `* * * * *` o expandir el rango (`0-59/1`).

**menu-procesos.sh → opción "listar"**

El script pide `ps` con formato propio (`-eo` en Linux / `-efo` en Solaris) para mostrar exactamente lo que pide la guía: **nombre, PID, %memoria, %CPU**.

**menu-procesos.sh → buscar / matar / reiniciar**

- **Buscar:** filtra `ps` por el nombre ingresado.
- **Matar:** `kill <PID>` → `SIGTERM`; `kill -9` → `SIGKILL`.
- **Reiniciar:** `kill -CONT <PID>` → `SIGCONT` (reanuda un proceso detenido).

**files-script.sh 10 1GB**

Se pasaron 2 argumentos → `$1` = 10 (cantidad), `$2` = 1GB (tamaño máximo).

Internamente: `find` recorre (con subdirectorios), `du` mide, `sort -n` ordena y `head -n 10` corta los 10 más pequeños.

## 5. EC2 — comando por comando

```
ssh -i lab03-key.pem ec2-user@ec2- ...
```

**Por qué pregunta "Are you sure you want to continue connecting":** es **TOFU** (*trust on first use*): la primera vez no conoce la huella del host, te la muestra para que la verifiques y la guarda en `known_hosts`. Si no coincide con la de la consola (*Get system log*), puede haber un **man-in-the-middle**.

**Por qué `ec2-user`:** es el usuario por defecto de la AMI de **Amazon Linux** (cada distro trae el suyo: `ubuntu` en Ubuntu, etc.).

### lsblk / montaje de un EBS

El volumen EBS se adjunta **en caliente** y el SO lo ve como un disco nuevo. Pasos:

```
mkfs.ext4 /dev/xvdf → mount → /etc/fstab para que persista.
```

### STOP vs TERMINATE vs RESTART

| Acción           | Qué pasa con el EBS raíz                                                    |
|------------------|-----------------------------------------------------------------------------|
| <b>Stop</b>      | La instancia se apaga; el EBS <b>persiste</b> ; se deja de facturar cómputo |
| <b>Terminate</b> | La instancia se <b>elimina</b> ; el EBS raíz se borra (salvo protección)    |
| <b>Restart</b>   | Reinicia el SO en la misma instancia (cambios de kernel/config)             |

**Ephemeral (instance store):** disco temporal ligado al host físico → **se pierde siempre** al detener/terminar.

## 6. Tabla resumen: "¿por qué veo esto?"

| Ves...                                          | Porque...                                                            |
|-------------------------------------------------|----------------------------------------------------------------------|
| <code>Server: 127.0.0.1</code>                  | El resolver está en la máquina local (punto 8)                       |
| <code>Server: 192.168.82.10</code>              | El cliente apunta al DNS de otra máquina (punto 7)                   |
| <code>Non-authoritative answer</code>           | Contestó un recursor desde su <b>caché</b> , no el dueño de la zona  |
| <code>No answer</code>                          | El nombre existe, pero no ese <b>tipo</b> de registro                |
| <code>NXDOMAIN</code>                           | El nombre <b>no existe</b> para ese servidor                         |
| <code>canonical name =</code>                   | Es un <b>CNAME</b> (alias → nombre real)                             |
| <code>named -u named</code>                     | El servicio <b>bajó privilegios</b> (no corre como root)             |
| <code>online</code> + <code>enabled=true</code> | Está activo <b>y</b> arranca al boot                                 |
| <code>UnKnown</code> (nslookup Windows)         | No hay registro inverso (PTR) del servidor DNS                       |
| <code>allow-transfer { ... }</code>             | Lista de <b>quiénes</b> pueden pedir la copia de mi zona (seguridad) |
| <code>masters { ... }</code>                    | De <b>quién</b> copio esta zona (soy esclavo)                        |
| Serial distinto entre maestro y esclavo         | El esclavo <b>no transfiere</b> : la copia quedó vieja               |